

Reviewer Pack — Coxeter Group Action and Circuit Depth Complexity

Entry b85359b3254a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 23:25:46 UTC

Coxeter Group Action and Circuit Depth Complexity

Entry ID: b85359b3254a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 23:25:46 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry **Field B** (complexity object): Boolean Circuits

Statement:

For every Boolean circuit C with depth d , the number of distinct Coxeter group actions required to construct C through a sequence of moves is at most 2^d .

Rationale (proposer's reasoning):

The action of Coxeter groups on polytopes and their connection to matrix Lie algebras provide a rich structure that could potentially be exploited in understanding circuit complexity. This conjecture suggests that the complexity of constructing circuits might be related to the number of group actions involved, offering a novel way to analyze circuit depth.

Taxonomy category: Combinatorial_Geometry_Boolean_Circuits (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dc4f6ac24d704d63

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The number of distinct Coxeter group actions required to construct a Boolean circuit C with depth d is at most 2^d .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group action" AND "Boolean circuit complexity" - "combinatorial geometry" AND "circuit depth" - "sequence of moves" AND "Coxeter group actions" AND "Boolean circuits"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=242.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(depth):
        if depth == 0:
            return ['NOT', 'XOR']
        else:
            op = random.choice(['AND', 'OR'])
            left = generate_circuit(depth - 1)
            right = generate_circuit(depth - 1)
            return [op, left, right]

    def count_distinct_actions(circuit):
        if isinstance(circuit, list):
            action = circuit[0]
            left = count_distinct_actions(circuit[1])
            right = count_distinct_actions(circuit[2])
            return {action} | left | right
        else:
            return set()

    def is_valid_circuit(circuit):
        if isinstance(circuit, list):
            op = circuit[0]
            left = circuit[1]
            right = circuit[2]
            if op not in ['AND', 'OR']:
                return False
            if not (isinstance(left, list) or isinstance(left, str)) or not (isinstance(right, list) or isinstance(right, str)):
                return False
            return is_valid_circuit(left) and is_valid_circuit(right)
        else:
            return circuit in ['NOT', 'XOR']
```

```

n_max = 40
instances_tested = 30
total_actions = 0

for _ in range(instances_tested):
    depth = random.randint(1, n_max)
    circuit = generate_circuit(depth)
    if not is_valid_circuit(circuit):
        continue

    actions = count_distinct_actions(circuit)
    total_actions += len(actions)

metric_value = total_actions / instances_tested
conjecture_holds = metric_value <= 2 ** n_max

return {
    "metric_name": "distinct_coxeter_group_actions",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_dev = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20472
2	preregistration	ollama_remote	glm4:latest	0	0	8707
3	novelty	ollama_remote	glm4:latest	0	0	9017
4	novelty	ollama_remote	glm4:latest	0	0	9005
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23331
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	71289
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14741
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	87151
9	judge	ollama_remote	glm4:latest	0	0	89394

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 333107 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/b85359b3254a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b85359b3254a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b85359b3254a.tar.gz (if generated)